Cryptography Main

A single, deduplicated, LLM-friendly compilation covering the history and foundations of cryptography, classical cryptanalysis, the formal/ mathematical basis of modern cryptography, symmetric and public-key primitives, protocols, advanced constructions, implementation security, and the socio-political dimensions of the field.

> **Sources merged (nothing repeated twice):**
>
> 1. Fabien Petitcolas' "Kerckhoffs Principle" dictionary entry + Auguste Kerckhoffs' original 1883 article *"La Cryptographie Militaire"* (Journal des sciences militaires) — historical foundations.
> 2. The full French original text of *"La Cryptographie Militaire,"* Part I (scanned/OCR'd PDF) — used to fill in taxonomy and biographical detail not present in the digest.
> 3. A prior compiled "Cryptography Master Reference" covering mathematical foundations through modern primitives, protocols, and socio-political context.
> 4. Jonathan Katz & Yehuda Lindell, *Introduction to Modern Cryptography* (2nd ed., CRC Press) — used only for its topic structure (formal security definitions, the provable-security paradigm) summarized in the compiler's own words; no text is reproduced from the book, in line with copyright limits.

Table of Contents

---

1. Historical Foundations — Kerckhoffs and Classical Cryptography

---

1.0 Who Was Kerckhoffs?

Auguste Kerckhoffs (1835–1903) was a Dutch-born linguist, trained at the University of Liège, who taught German at the École des Hautes Études Commerciales and the École Arago in Paris. Alongside a side interest in constructed languages (he supported Volapük), he had a serious interest in cryptography. In January and February 1883 he published a two-part article, *"La Cryptographie Militaire,"* in the *Journal des sciences militaires* (vol. IX, pp. 5–38 and pp. 161–191), surveying the cryptographic methods of his day and setting out design principles for military ciphers. This article is the origin of what is now called **Kerckhoffs' Principle**.

1.1 The Six Principles for a Military Cipher

Kerckhoffs distinguished a cipher meant for a one-off private exchange (where almost any clever scheme will do) from one meant for indefinite, ongoing military use, which he argued must satisfy six conditions:

1. **Practical unbreakability** — the system must be unbreakable in practice, if not in a strict mathematical sense.
2. **No requirement of secrecy** — the system must not depend on being secret, and it should cause no harm if it falls into enemy hands. *(This is the modern "Kerckhoffs' Principle.")*
3. **Memorable, changeable keys** — the key must be short enough to memorize (no written notes needed) and easy for correspondents to change or agree on.
4. **Telegraph-compatible** — the cipher must work with telegraphic transmission.
5. **Portable, single-operator** — the apparatus must be portable and usable by one person alone.
6. **Simple to use** — given the stressful conditions of military use, the system must not require mental strain or a long list of rules.

Kerckhoffs treated the last three (4–6) as uncontroversial. His real argument was about the first three, and especially about Principle 2.

The core argument (Principle 2). Kerckhoffs contended that requiring secrecy of the *method* itself (as opposed to just the key) was the fundamental weakness of every cryptographic system used by the French army of his day:

- A secret system can only safely be entrusted to a small number of people, so its use is effectively restricted to high commanders — it cannot be pushed down to regimental or column commanders, which is where he thought secret communication was actually most needed.
- A secret method is vulnerable at every "engagement" where it might be captured, exposing the whole army's communications, not just one message.
- He rejected the contemporary argument that a message only needs to stay secret for a few hours ("it'll be stale by the time they read it"): long sieges and multi-day operations mean secrecy often

needs to last much longer, and once one intercepted cryptogram is broken, every future message using the same key becomes instantly readable.
- Mathematically perfect (absolute) unbreakability is impossible, but *practical* unbreakability combined with total openness about the method is achievable.
- His policy recommendation: the War Ministry should abandon secret methods entirely and adopt only a system that could be taught openly in military schools, freely shared with anyone, and even copied by rival nations — because true security should come only from the key.

The generalized idea. Kerckhoffs' 2nd and 3rd principles generalize into a broader security philosophy: **a secret is a point of failure.** Secrets should be minimized in number and made easy to replace when compromised — a principle now applied throughout security engineering, not just cryptography.

Shannon's Maxim. In the 20th century, Claude Shannon gave the information-theoretic grounding for why this works, restating the same idea concisely as **"the enemy knows the system."** This restatement is now called **Shannon's Maxim** and is treated as functionally equivalent to Kerckhoffs' Principle.

Kerckhoffs' Principle vs. "security through obscurity." Kerckhoffs' Principle and Shannon's Maxim stand in direct opposition to *security through obscurity* — the idea that hiding *how* a system works is itself a valid layer of protection. Modern cryptographic design treats obscurity of the algorithm as providing no real security margin: only the secrecy and strength of the key should matter. This remains one of the central design tenets of modern cryptosystems (see §17–18 for how it shapes standards and policy debates today).

1.2 Cipher Taxonomy (as surveyed by Kerckhoffs, 1883)

Kerckhoffs organized 19th-century ciphers into three families, illustrated throughout with worked examples from real intercepted correspondence:

A. Transposition ciphers — rearrange the letters of the plaintext

without changing them.

- *Simple columnar transposition:* write the plaintext into rows of fixed width, then read it off by a key-determined column order (the key is a word converted to a number sequence, e.g. "Schuvalow" → 6 2 3 7 8 1 4 5 9).
- *Double transposition:* transposing first by columns, then again by rows, usually with the same keyword. Kerckhoffs showed contemporary Russian nihilist ciphers using this scheme were broken quickly once the method was suspected, because certain letters (French *q*, always followed by *u*; *x*, usually preceded by *u*) only occur in predictable combinations and give the transposition away.
- *Grille (grid) ciphers:* mechanical devices with cut-out cells rotated through four positions over a numbered grid (invented by Cardano in the 16th century); Colonel Fleissner's rotating-grille refinement (*neue Patronen-Geheimschrift*, 1881) was considered close to indecipherable but, like all secret-device schemes, violated Principle 2 by requiring the device itself to stay secret.

B. Substitution ("interversion") ciphers — replace each plaintext letter with another symbol.

- *Single-alphabet (monoalphabetic) ciphers:* one fixed substitution alphabet for the whole message. Includes the classical Caesar shift (Julius Caesar shifted 3–4 ranks; Augustus used a similar scheme for family letters). Kerckhoffs judged these to offer *no real security*, since a cryptogram of even two or three lines can be solved almost by inspection using letter-frequency and bigram/trigram analysis (§1.3).
- *Polyalphabetic (double-key) ciphers:* the substitution alphabet changes per letter, based on a repeating keyword. Kerckhoffs covered:
    - **Porta's cipher** (Giambattista della Porta, 1563) — the first literal polyalphabetic cipher, using 11 paired alphabets (AB, CD, EF, …) selected letter-by-letter via a keyword; Kerckhoffs credits Porta, not Trithemius, as the true founder of cryptography, since Porta was first to introduce a genuine key into a letter-substitution scheme.
    - **Vigenère's tableau ("chiffre carré" / square cipher)** —

Blaise de Vigenère's 1586 simplification of Porta's system using a 26×26 tabula recta (one row per possible shift). Widely — and wrongly — believed to be unbreakable ("le chiffre indéchiffrable"); still in use by the French War Ministry after 1870.
    - **Saint-Cyr system** — a portable, disguised version of the Vigenère tableau (two sliding paper alphabet strips) taught at the French military school of Saint-Cyr; mathematically identical to Vigenère and producing an identical cryptogram for the same key.
    - **Beaufort cipher** — Admiral Francis Beaufort's variant (1857) that alters the lookup direction on the tableau, giving a text an unfamiliar, "unbreakable" look to the uninitiated; Kerckhoffs shows it's exactly equivalent to a Vigenère/Saint-Cyr cipher run against a reversed alphabet.
    - **Gronsfeld cipher** — a numeric-keyed variant of Vigenère (the key is a number, e.g. 102, not a word), doable mentally without a physical table; Kerckhoffs judged it no harder to break than the others, and a "modeste chiffre à simple clef" in disguise.
    - **Variable-key ("running") system** — disguises the key length by breaking the regular repetition of the keyword at irregular points, using a designated "stop letter" (e.g., key *Epaminondas* broken into *epa + epaminondas… + epami + epaminon…*) whose recurrence pattern can nonetheless be detected because it never appears twice in a row.

C. Coded dictionaries ("dictionnaires chiffrés" / nomenclators) — whole words, syllables, or phrases replaced by numeric codes from a codebook, sometimes with an added permutation-plus-addition step (Brunswick and Gallian's "double-combination" codebooks) to further disguise the raw code numbers. Kerckhoffs judged these among the *most* practically secure systems of the era, but criticized them heavily because:

- They depend on absolute physical secrecy of the codebook — an outright violation of Principle 2.
- A single mistyped digit in a telegram can silently and completely change the meaning of an order (Kerckhoffs cites several historical mishaps, including a Ministry cryptogram that could not

be decoded on a Sunday because the one officer who held the key was absent, and armies rendered unable to read urgent dispatches when codebooks were lost, inaccessible, or left behind — cases the Master Reference identifies as the Franco-Prussian War of 1871 and the Russo-Turkish War of 1877).

1.3 Cryptographic Devices ("Cryptographes")

Kerckhoffs reviewed the mechanical cipher devices of his era and assessed their real security value:

- **Ancient devices:** the Spartan scytale, Aeneas Tacticus's perforated board, "Kessler's barrel" (1616).
- **Porta's disk cipher** (1563) — the origin of the rotating-disk substitution idea, later reused in Wheatstone's cryptograph and various dial-based devices sold during the Italian war of 1859.
- **Mouilleron's device** — shown to be mathematically just a Vigenère cipher in mechanical form, despite being marketed as novel.
- **Vinay and Gaussin's device** — a combined printer/encryption mechanism; too bulky for field use and no stronger cryptographically than existing methods.
- **Rondepierre's "phyrographe"** — an ivory-rod transposition device; practical but only offers relative security.
- **Silas's cryptograph** — well engineered but too complex for battlefield use.
- **Wheatstone's cryptograph** — judged the best of the newer devices for simplicity: a keyword-derived cipher alphabet on a rotating dial with two hands moving at different speeds. Kerckhoffs nonetheless shows it is fully breakable — once the alphabet's "cycle length" is known, it reduces to the same column-based analysis as any polyalphabetic cipher — and it still requires total secrecy of the device to have any value, again violating Principle 2.

1.4 Breaking Ciphers — Kerckhoffs' Cryptanalysis Techniques

Kerckhoffs laid out concrete, step-by-step cryptanalytic techniques current in the 1880s — an early practical treatise on the subject that predates and, in places, anticipates the modern Kasiski examination (published by Major Kasiski in 1863, which Kerckhoffs considered unnecessarily complicated).

- **Letter-frequency analysis:** in French, roughly one letter in five is "e"; Kerckhoffs gives full letter-frequency tables from real French military correspondence (E=185, S=88, R=78, I=74, A=72 per 1,000 letters, etc.) and notes the most frequent letter differs by language — E for French/English/German, O for Spanish, A for Russian, E/I for Italian.
- **Breaking monoalphabetic substitution:** identify the most frequent cipher symbol as "e," then use common bigram/trigram patterns (*es, en, se, te, et, de, me, el, em, le* in French) plus word-structure reasoning to reconstruct the rest of the alphabet. Kerckhoffs demonstrates this on a worked cryptogram, deducing the French sentence *"Il a été difficile de lire ses lettres"* letter-group by letter-group from trigram/monogram constraints alone.
- **Breaking polyalphabetic ("double-key") ciphers** — Kerckhoffs' most technically original contribution:
    1. **Determine key length:** find repeated letter groups (bigrams/trigrams) in the ciphertext; the distances between repeats share a common factor, which reveals the number of alphabets (the key length). This is essentially the same insight underlying the modern **Kasiski examination**.
    2. **Determine alphabet ordering:** split the ciphertext into columns by key-letter position, then apply frequency analysis independently to each column (each column is just a monoalphabetic cipher).
    3. **"Symmetry of position"** — Kerckhoffs' own named technique, which he claims to be first to publish: if the substitution alphabets are all shifts of one "square" table (as in Vigenère/Saint-Cyr/Gronsfeld), discovering the mapping in one alphabet instantly reveals related letter mappings in every other alphabet, because relative letter spacing is preserved across all 26 shifted alphabets.
    4. **Context/linguistic reasoning:** worked example decrypting a real intercepted 1882 Reuters/Havas telegram about the British campaign in Egypt (Wolseley, Ismaïlia, Suez),

combining educated guesses about likely opening words with partial frequency data to reconstruct the message word by word.
    5. **Multiple ciphertexts under one key:** if several short messages share a key, stacking them lets the repeated-letter-group technique be applied across messages instead of within one, greatly easing key-length recovery even for very short texts.
- **Breaking coded dictionaries:** shown via a worked numeric example — guessing the plaintext meaning of just two intercepted code groups and comparing the assumed vs. actual numbers can reveal the underlying permutation formula and additive key, after which the rest of the dictionary-coded message becomes readable.

1.5 Kerckhoffs' Closing Thesis and Legacy

Kerckhoffs closed the article by generalizing his critique: transposition-only systems are secure only if the transposition mechanism itself is secret; multi-alphabet substitution systems are breakable if regular/systematic, and impractical if made complex enough to resist his own analytical methods; coded dictionaries are the most secure in practice but violate the "no secrecy required" desideratum outright. He proposed — mostly as a curiosity, admitting it was impractical for real battlefield use — a **triple-key system** combining Saint-Cyr-style substitution with columnar transposition under two independent keywords.

He ended by endorsing a maxim from the 17th-century cryptographer Jean Robert du Carlet as the ultimate design standard for any military cipher:

> **"Ars ipsi secreta magistro"** — a cipher is only good if it remains undecipherable even to the master who invented it.

This is the historical ancestor of the modern practice of subjecting cryptographic algorithms to public, adversarial scrutiny rather than relying on any one party's — including the designer's — confidence in its secrecy. Writing nearly a century later, Phillip Rogaway's "The Moral Character of Cryptographic Work" (see §18.1) echoes the same insight

in modern form: a cipher's *design* must never be allowed to become an institutional point of vulnerability.

Summary table: then vs. now

Kerckhoffs (1883)	Modern equivalent
Principle 2: system must not require secrecy of the method	Kerckhoffs' Principle — algorithms should be public; only keys are secret
"The enemy knows the system"	Shannon's Maxim
Rejection of secret dictionaries/mechanisms as a security basis	Rejection of "security through obscurity"
Symmetry-of-position technique for breaking Vigenère-family ciphers	Precursor to the Kasiski examination / modern polyalphabetic cryptanalysis
"Ars ipsi secreta magistro" (du Carlet, 1644)	Open, peer-reviewed cryptographic design (e.g., public algorithm competitions like AES/SHA-3)
Letter-frequency & repeated-group analysis	Index of Coincidence and classical cryptanalysis (see §14.1)

---

2. Mathematical and Theoretical Foundations

2.1 Number Theory

- **Modular Arithmetic:** foundation of public-key cryptography; operations mod *n* create finite rings/fields.
- **Prime Numbers:** central to RSA and Diffie-Hellman; primality testing via Miller-Rabin, AKS.
- **Discrete Logarithm Problem (DLP):** given *g* and *g^x mod p*,

finding *x* is infeasible for large primes. Underpins Diffie-Hellman, ElGamal, ECC.
- **Integer Factorization:** given $n = p \cdot q$, finding *p, q* is hard. Underpins RSA.
- **Euclidean Algorithm:** computes gcd(a,b). The Extended Euclidean Algorithm derives Bézout coefficients *s, t* satisfying $sa + tb = gcd(a,b)$, enabling modular inverses $a^{-1} \bmod m$.
- **Modular Exponentiation:** repeated squaring achieves O(log e) multiplications for $x^e \bmod m$.

2.2 Abstract Algebra

- **Groups:** set with associative operation, identity, inverses. Cyclic groups of prime order underlie discrete-log cryptography.
- **Rings and Fields:** finite fields F_p (prime fields) and F_(2^m) (binary extension fields) underlie AES and ECC respectively.
- **Galois Field Arithmetic F_(2^m):** elements represented as polynomials $A(x) = \Sigma\, a_i x^i \bmod P(x)$. AES uses irreducible polynomial $P(x) = x^8 + x^4 + x^3 + x + 1$. Inversion uses the Extended Euclidean Algorithm on polynomial rings.
- **Elliptic Curves:** $y^2 = x^3 + ax + b \bmod p$ over F_p, with non-singularity condition $4a^3 + 27b^2 \neq 0 \bmod p$. Points form an abelian group under geometric addition.
- **Lattice Theory:** a lattice $\Lambda \subset R^n$ is generated by integer combinations of basis vectors $B = \{b_1,...,b_k\}$. Key concepts:
  - Fundamental Parallelepiped: $P(B) = \{\Sigma x_i b_i \mid 0 \leq x_i < 1\}$
  - Lattice Volume: $\det(\Lambda) = \sqrt{\det(B^T B)}$
  - Unimodular Transformation: basis equivalence via integer matrices U with $\det(U) = \pm 1$
  - Successive Minima $\lambda_i(\Lambda)$: smallest radius containing *i* linearly independent lattice vectors. Minkowski's Theorem: $\lambda_1(\Lambda) \leq \sqrt{n} \cdot \det(\Lambda)^{(1/n)}$.

2.3 Complexity Theory

- **One-Way Functions (OWFs):** $f:\{0,1\} \rightarrow \{0,1\}$ computable in polynomial time, where no probabilistic polynomial-time (PPT) algorithm can find a preimage except with negligible probability.

- **Goldreich-Levin Theorem:** for any OWF f, the inner product $B_r(x) = \langle x,r \rangle \bmod 2$ is a hard-core predicate — unpredictable from (f(x), r) better than 1/2 probability.
- **Blum-Micali Construction:** any OWF with a hard-core predicate can be iteratively evaluated to stretch a random seed into a cryptographically secure pseudorandom sequence.
- **NP-Complete Problems:** used in interactive proofs; Graph Three-Colorability is the canonical zero-knowledge example (see §8.2).

2.4 Probability and Information Theory

- **Shannon's Perfect Secrecy:** P[M=m | C=c] = P[M=m] — observing the ciphertext gives an adversary no information at all about the message. The One-Time Pad is the only perfectly secure cipher (key as long as message, used once, truly random).
- **Information-Theoretic vs. Computational Security:** the former holds against unbounded adversaries; the latter relies on assumed problem hardness and bounded (polynomial-time) adversaries.
- **Entropy:** measures uncertainty; high-entropy sources are essential for key generation. Low-entropy key derivation is a common real-world vulnerability (see §13.6).

2.5 Formal Security Definitions and the Provable-Security Paradigm

Modern (post-1980s) cryptography is distinguished from the classical, ad-hoc cipher design surveyed in §1 by its insistence on three disciplines: precise **definitions** of what "secure" means, explicit **assumptions** about what problems are hard, and mathematical **proofs of security** that reduce a scheme's security to those assumptions. This shift — from "does anyone see how to break it?" to "can we prove no efficient adversary can break it, assuming problem X is hard?" — is the central methodological difference between the Kerckhoffs-era ciphers of §1 and everything in §3 onward.

- **Security as a game, not a property:** a scheme is defined secure relative to an explicit adversarial "experiment" (game) in which a

probabilistic polynomial-time adversary interacts with the scheme and must win with only negligible advantage over guessing.

- **Semantic security:** informally, ciphertexts leak no partial information about the plaintext to any efficient adversary, no matter what side information the adversary has — the encryption-scheme analogue of perfect secrecy, adapted to computationally bounded adversaries.
- **Indistinguishability (IND) games — the practical formulation of semantic security:**
  - **IND-CPA (chosen-plaintext attack):** the adversary may request encryptions of plaintexts of its choice, then must distinguish the encryption of one of two chosen plaintexts from the other, with negligible advantage over ½.
  - **IND-CCA1 (lunchtime attack):** the adversary additionally gets decryption-oracle access *before* seeing the challenge ciphertext.
  - **IND-CCA2 (adaptive chosen-ciphertext attack):** the adversary retains decryption-oracle access even *after* seeing the challenge ciphertext (barred only from querying the challenge itself) — the strongest standard notion, matching real-world attacks like padding-oracle attacks (§3.1, §14.3).
- **Building blocks with formal definitions:**
  - **Pseudorandom Generator (PRG):** stretches a short truly-random seed into a longer string computationally indistinguishable from true randomness.
  - **Pseudorandom Function (PRF):** a keyed function family indistinguishable, to any efficient adversary with oracle access, from a truly random function.
  - **Pseudorandom Permutation (PRP):** a PRF that is also efficiently invertible given the key — the formal model for a block cipher (§3.1). "Strong" PRP security also requires indistinguishability when the adversary can query the inverse direction.
- **Proof by reduction:** the standard proof technique — assume a hypothetical adversary that breaks the scheme's security game, then show how to use that adversary as a subroutine to solve the underlying hard problem (e.g., distinguish a PRG's output from random, or factor an RSA modulus). Since the hard problem is assumed infeasible, no such adversary can exist. The **hybrid**

argument is the standard tool for such proofs: a sequence of intermediate ("hybrid") distributions is constructed between the real and ideal experiments, and adjacent hybrids are shown indistinguishable one step at a time.

- **Message authentication and signatures:** the analogous strong security notion is **existential unforgeability under chosen-message attack (EU-CMA)** — an adversary with oracle access to the MAC/signing algorithm on messages of its choice still cannot produce a valid tag/signature on *any* new message it didn't query.
- **The KEM/DEM (hybrid encryption) paradigm:** public-key operations are slow, so practical public-key encryption combines a Key Encapsulation Mechanism (uses the public key just to establish a fresh symmetric session key) with a Data Encapsulation Mechanism (a fast symmetric AEAD scheme, §3.3, that actually encrypts the message) — this is the structural basis of TLS (§7.1) and most real-world hybrid encryption.
- **Provable security vs. real-world security:** a security proof only guarantees that breaking the scheme is at least as hard as breaking the underlying assumption *within the model as defined* — it does not cover implementation flaws, side channels (§13), protocol-composition errors, or misuse. This gap between mathematical proof and real-world deployment is a recurring theme throughout §13–14 and §18 below.

---

3. Symmetric Cryptography

3.1 Block Ciphers

AES (Advanced Encryption Standard)

- Designers: Joan Daemen and Vincent Rijmen (Rijndael); Standard: NIST FIPS 197 (2001).
- Block size 128 bits; keys 128/192/256 bits; rounds 10/12/14 respectively.
- Structure: Substitution-Permutation Network (SPN); operations:

SubBytes, ShiftRows, MixColumns, AddRoundKey.
- Formally modeled as a keyed Pseudorandom Permutation (§2.5); no practical attacks on full AES exist — related-key attacks apply only to reduced-round variants.
- Hardware acceleration: AES-NI (Intel/AMD), ARMv8 Crypto Extension.

DES / 3DES

- DES: 64-bit block, 56-bit effective key, 16 rounds — broken by brute force (Deep Crack, 1998).
- 3DES: applies DES three times; 112/168-bit keys; vulnerable to SWEET32 (64-bit block); deprecated by NIST.

Modes of Operation

Mode	Description	Properties	Security Notes
ECB	Independent block encryption	Parallelizable, deterministic	Insecure — reveals plaintext patterns
CBC	XORs previous ciphertext block	Sequential decryption	Needs unique IV; padding-oracle risk
CTR	Encrypts counter as keystream	Parallelizable, random access	Secure with unique nonce
GCM	CTR + GHASH authentication	AEAD, parallelizable	Catastrophic failure on nonce reuse
CCM	Counter + CBC-MAC	AEAD, constrained environments	Slower than GCM, two passes
XTS	Tweaked codebook + ciphertext stealing	Disk encryption, per-sector tweak	No authentication

Other Block Ciphers: Blowfish/Twofish (Schneier; Twofish was an AES finalist), Serpent (AES finalist, conservative margins), Camellia (Japan),

ARIA (South Korea).

3.2 Stream Ciphers

ChaCha20 / Salsa20

- Designer: Daniel J. Bernstein (djb); ARX structure (Add-Rotate-XOR) on 32-bit words.
- Key 256 bits; nonce 96 bits (XChaCha20: 192 bits); 20 rounds.
- No lookup tables → inherently constant-time; fast without hardware acceleration.
- RFC 8439; used in TLS 1.3, WireGuard, Signal.

Other Stream Ciphers: RC4 (broken, deprecated), A5/1 & A5/2 (GSM, both broken/weak), E0 (Bluetooth, vulnerable to correlation attacks), Grain/Trivium (eSTREAM; Trivium uses 288-bit state across 3 NLFSRs).

- **Linear Feedback Shift Registers (LFSRs):** m-stage LFSRs governed by feedback polynomial $P(x) = 1 + c_1x + ... + c_mx^m$ over F_2. Vulnerable to known-plaintext attacks via matrix solving (2m plaintexts recover the feedback polynomial).

3.3 Authenticated Encryption (AEAD)

Provides confidentiality + authenticity together (an AEAD scheme is IND-CCA2 secure by construction when built correctly, §2.5); associated data is authenticated but not encrypted.

Constructions: **AES-GCM** (widely deployed, careful nonce management required), **AES-CCM** (Wi-Fi WPA2/3, Bluetooth LE), **ChaCha20-Poly1305** (preferred without AES-NI, constant-time by design), **AES-GCM-SIV** (synthetic IV, robust against nonce reuse).

ChaCha20-Poly1305 vs AES-256-GCM

Feature	ChaCha20-Poly1305	AES-256-GCM
Type	Stream cipher + MAC	Block cipher (CTR) +

Key size	256 bits	128/192/256 bits
Nonce	96 bits (XChaCha20: 192)	96 bits
Auth tag	128 bits (Poly1305)	128 bits (GHASH)
HW acceleration	None needed	AES-NI, ARMv8 CE
Constant-time	Yes by design	Implementation-depe
FIPS 140-2	No	Yes
TLS 1.3 suite	TLS_CHACHA20_POLY1305_SHA256	TLS_AES_256_GCM
Speed, no AES-NI	~4,200 MB/s	~1,800 MB/s
Speed, with AES-NI	~396–1,158 MB/s	~577–2,617 MB/s
Post-quantum margin	~128-bit	~128-bit

4. Cryptographic Hash Functions

4.1 Required Properties

1. **Preimage resistance:** given y, hard to find x with H(x)=y.
2. **Second-preimage resistance:** given x, hard to find x'≠x with H(x')=H(x).
3. **Collision resistance:** hard to find any x≠x' with H(x)=H(x').

4.2 Hash Function Families

SHA-2 (FIPS 180-4): SHA-256 (256-bit, most widely used), SHA-384/SHA-512 (SHA-512 gives 256-bit collision resistance), truncated variants (SHA-224, SHA-512/224, SHA-512/256). Merkle-Damgård construction with Davies-Meyer compression. ~1.2 GB/s single-core, sequential only.

SHA-3 / **Keccak (FIPS 202):** 2012 NIST competition winner. Sponge construction (not Merkle-Damgård). Variants: SHA3-224/256/384/512, SHAKE128/256 (XOFs). Resistant to length-extension attacks. ~0.8 GB/s, not parallelizable.

BLAKE Family:

- BLAKE2 (2012): faster than MD5 at SHA-3-equivalent security; BLAKE2b (64-bit platforms), BLAKE2s (32-bit); built-in keyed hashing (no HMAC needed); used in Linux kernel RNG, WireGuard, Argon2.
- BLAKE3 (2020, Aumasson/O'Connor/Neves/Wilcox-O'Hearn): Merkle tree over 1 KiB chunks + BLAKE2s-derived compression; parallel scaling (~1 GB/s single-core to 30+ GB/s on 32 cores); modes for hashing, MAC, and KDF; not yet FIPS.

Broken (historical): MD5 (128-bit, practical collisions since 2004), SHA-1 (160-bit, SHAttered collision 2017, chosen-prefix collisions demonstrated) — both deprecated.

4.3 Performance Comparison

Algorithm	Speed (GB/s)	Output	Security Level	Status
MD5	~0.7	128 bits	Broken	Do not use
SHA-1	~0.9	160 bits	Broken	Deprecated
SHA-256	~1.2	256 bits	128-bit	Standard
SHA-512	~1.8	512 bits	256-bit	Standard
SHA3-256	~0.8	256 bits	128-bit	Standard
BLAKE2b	~2.0	up to 512 bits	128-bit	Widely used

BLAKE3	~6.0+	256 bits (extendable)	128-bit	Modern choice

4.4 Hash-Based Constructions

- **HMAC:** HMAC(K,m) = H((K'⊕opad) ‖ H((K'⊕ipad) ‖ m)); provably secure under reasonable hash assumptions (FIPS 198-1, RFC 2104).
- **HKDF:** Extract-then-Expand key derivation (RFC 5869); used in TLS 1.3, Signal.
- **Merkle Trees:** binary tree of hashes for efficient verification of large datasets; used in blockchain, certificate transparency, file integrity.

---

5. Message Authentication Codes (MACs)

- **HMAC:** uses a hash function (typically SHA-256); most widely deployed MAC; the practical instantiation of the EU-CMA notion described in §2.5.
- **CMAC:** uses a block cipher (typically AES); NIST SP 800-38B; no hash function needed.
- **GMAC:** authentication component of AES-GCM; uses GHASH over GF(2^128); requires unique nonce.
- **Poly1305:** one-time MAC by Bernstein; extremely fast in software; paired with ChaCha20 (TLS 1.3, WireGuard); key must be unique per message.
- **KMAC:** based on SHA-3/Keccak's cSHAKE (NIST SP 800-185); variable output length.

---

6. Public Key Cryptography

6.1 RSA

Foundation: integer factorization.

Key generation: choose primes p,q → n=pq, φ(n)=(p-1)(q-1) → choose e (typically 65537) → d = e^{-1} mod φ(n) → public (n,e), private (n,d).

Operations: Encrypt c=m^e mod n; Decrypt m=c^d mod n; Sign s=m^d mod n; Verify m=s^e mod n.

Security notes:

- Minimum key size 2048 bits (3072+ for long-term security).
- Raw ("plain") RSA is deterministic and not IND-CPA secure (§2.5) — padding is essential.
- RSA-OAEP: CCA2-secure encryption via multi-stage hash padding (Random Oracle Model).
- RSA-PSS: probabilistic signatures with randomized salts.
- Bleichenbacher attacks: padding-oracle attacks on PKCS#1 v1.5.
- ROCA (2017): weak randomness in Infineon TPM RSA key generation.
- Common Modulus Attack: never share n between key pairs.
- Small private exponent attacks: Wiener's and Boneh-Durfee attacks when d < n^0.292.

6.2 Diffie-Hellman Key Exchange

Foundation: Discrete Logarithm Problem.

Protocol: agree on p, g → Alice sends A=g^a mod p → Bob sends B=g^b mod p → shared secret s = B^a = A^b = g^(ab) mod p.

Security notes: Logjam (2015, downgrade to 512-bit export DH), FREAK (2015, similar RSA export downgrade), safe-prime requirement p=2q+1, minimum 2048-bit primes (3072+ recommended), ECDH as the more efficient elliptic-curve variant.

6.3 Elliptic Curve Cryptography (ECC)

Advantage: RSA-equivalent security at much smaller key sizes.

Curve types: Weierstrass ($y^2=x^3+ax+b \bmod p$), Montgomery ($By^2=x^3+Ax^2+x$, fast scalar multiplication), Edwards ($x^2+y^2=1+dx^2y^2$, complete addition formulas).

Standard curves: NIST P-256/P-384/P-521 (widely used, criticized for opaque parameter generation), Curve25519/X25519 (djb, 128-bit security, fast/constant-time), Curve448/X448 (224-bit margin), Ed25519 (signatures), secp256k1 (Bitcoin, Koblitz curve), BLS12-381 and BN254 (pairing-friendly, used in ZK-SNARKs).

Point addition ($P=(x_1,y_1)$, $Q=(x_2,y_2)$, $P\neq Q$):

- slope $m = (y_2-y_1)/(x_2-x_1) \bmod p$ (doubling: $m = (3x_1^2+a)/(2y_1) \bmod p$)
- $x_3 = m^2 - x_1 - x_2 \bmod p$
- $y_3 = m(x_1-x_3) - y_1 \bmod p$

6.4 Digital Signature Algorithms

ECDSA

- Domain: prime p, curve coefficients a,b, generator G, subgroup order n, cofactor h.
- Sign: pick random $k \in [1,n-1]$; $(x_1,y_1)=kG$; $r=x_1 \bmod n$; $s=k^{-1}(z+rd) \bmod n$; signature=(r,s).
- Verify: $u_1=zs^{-1} \bmod n$, $u_2=rs^{-1} \bmod n$; $P=u_1G+u_2H\_A$; valid iff $x\_P \bmod n == r$.
- **Critical vulnerability:** reusing nonce k across two messages under the same key allows private-key extraction via linear algebra.
- **Malleability:** (r,s) can be rewritten as (r, -s mod L) without the private key — requires strict normalization.

EdDSA (Ed25519/Ed448) — djb et al. Deterministic nonce $k=H(\text{private\_key} \| \text{message})$ removes reliance on external entropy; fast constant-time; no malleability; compact 64-byte signatures. Ed25519: 128-bit security, ~30,000× faster verification than RSA-2048. Ed448: 224-bit margin.

DSA: predecessor to ECDSA using finite-field discrete logs; deprecated in favor of ECDSA/EdDSA.

6.5 Key Encapsulation Mechanisms (KEMs)

- Classical: RSA-KEM (no forward secrecy), ECDH-based KEMs (forward secrecy).
- Post-quantum: ML-KEM/CRYSTALS-Kyber (FIPS 203, lattice-based, 3 parameter sets), HQC (NIST-selected additional code-based KEM, March 2025).
- See §2.5 for the general KEM/DEM hybrid-encryption paradigm these are built for.

---

7. Key Exchange and Network Protocols

7.1 TLS

TLS 1.2 (RFC 5246, 2008) — 2-RTT handshake: Client Hello → Server Hello/certificate → key exchange (pre-master secret encrypted with server's public key) → both derive session keys → Change Cipher Spec + Finished. Issues: 100+ possible (often insecure) cipher suites, optional forward secrecy, downgrade-vulnerable version negotiation, compression enabled (CRIME/BREACH).

TLS 1.3 (RFC 8446, 2018) — 1-RTT handshake (0-RTT resumption with replay risk); AEAD-only cipher suites; mandatory forward secrecy (ephemeral DH); removed RSA key exchange, static DH, MD5, SHA-1, RC4, 3DES, compression; modern signatures (ECDSA, RSA-PSS; DSA removed); more of the handshake encrypted; explicit downgrade protection in ServerHello.random.

Cipher suites: TLS_AES_256_GCM_SHA384, TLS_CHACHA20_POLY1305_SHA256, TLS_AES_128_GCM_SHA256, TLS_AES_128_CCM_SHA256, TLS_AES_128_CCM_8_SHA256. As of January 2024, NIST requires TLS 1.3 support.

7.2 IPsec

- **IKEv2:** IKE_SA_INIT (algorithm agreement, DH shared secret) → IKE_AUTH (identity validation via certificates or PSKs).
- **Encapsulation:** AH (integrity/origin authentication, no confidentiality) vs. ESP (confidentiality + integrity + anti-replay).
- **Key management:** OAKLEY standardizes key establishment across certificates, PSKs, and third-party authorities.

7.3 Signal Protocol / Double Ratchet

- **X3DH:** initial key agreement combining multiple DH operations.
- **Double Ratchet:** Symmetric ratchet (KDF chain of message keys) + DH ratchet (asymmetric update each reply → future secrecy).
- Properties: forward secrecy, future secrecy (self-healing), deniability.
- Used in Signal, WhatsApp, Facebook Messenger.

7.4 Noise Protocol Framework

Framework (not a single protocol) for building crypto protocols via named handshake patterns (e.g., Noise_XX, Noise_IK) describing initiator/responder static/ephemeral key usage. Used in WireGuard, Lightning Network, WhatsApp. Minimal, formally verified, no legacy baggage.

7.5 WireGuard

Modern VPN using Noise_IK. Crypto stack: Curve25519 (ECDH), ChaCha20-Poly1305 (AEAD), BLAKE2s (hashing), SipHash24 (hashtable keys). ~4,000 lines of code vs. 400,000+ for IPsec/ OpenVPN; kernel-space; fast roaming.

---

8. Zero-Knowledge Proof Systems

8.1 Foundational Concepts

Introduced by Goldwasser, Micali, and Rackoff (1980s): a Prover (P) convinces a Verifier (V) that a statement is true, revealing nothing beyond that single bit of validity.

Three required properties:

1. **Completeness** — an honest Prover convinces an honest Verifier with probability 1.
2. **Soundness** — no cheating Prover can convince an honest Verifier of a false statement except with negligible probability.
3. **Zero-knowledgeness** — formalized via a **Simulator (Sim)** that, without the witness, produces a transcript indistinguishable from a real interaction.

8.2 Interactive Proof Example: Graph Three-Colorability

Prover claims a valid 3-coloring of G=(V,E):

1. Prover randomly permutes colors, commits to each vertex's color.
2. Verifier randomly picks an edge e=(u,v) $\in$ E.
3. Prover reveals the commitments for u and v.
4. Verifier checks the colors are distinct.

Cheating probability per round ≤ 1 – 1/|E|; after k rounds, Prob_cheat ≤ (1 – 1/|E|)^k. E.g., |E|=1000 with k=5000 rounds drives soundness error to negligible levels.

Proof vs. Proof of Knowledge: a Proof asserts a statement is true; a Proof of Knowledge asserts the Prover actually possesses a specific witness.

8.3 Schnorr Identification Protocol

Over cyclic group G, prime order q, generator g:

1. Prover picks random $r \in Z_q$, sends $r' = g^r \bmod p$.
2. Verifier sends random challenge $c \in Z_q$.
3. Prover computes $s = r + ca \bmod q$, sends s. Verifier checks $g^s \equiv r' \cdot y^c \bmod p$.

Knowledge Extractor: rewinds the Prover to obtain two responses s_1, s_2 for two challenges c_1, c_2 on the same commitment: $s_1 = r + c_1 a$, $s_2 = r + c_2 a \pmod q \implies a = (s_1 - s_2)(c_1 - c_2)^{-1} \bmod q$. This proves any Prover answering two distinct challenges must know the secret.

8.4 Non-Interactive ZK

Fiat-Shamir Transformation: converts interactive sigma protocols into non-interactive signatures by replacing the Verifier's challenge with a hash of the commitment and message.

8.5 Modern ZK Proof System Comparison

Parameter	ZK-SNARKs	ZK-STARKs	Bulletproofs
Setup	Trusted Setup (SRS) via MPC ceremony	Fully transparent	Fully transparent
Primitives	Bilinear pairings (BN254, BLS12-381), KZG commitments	FRI + hash functions	Inner Product Args, Vector Pedersen Commitments
Prover complexity	O(N log N)	O(N·polylog N)	O(N log N)
Verifier complexity	O(1)	O(polylog N)	O(N) linear
Proof size	~200–500 bytes	~10–100 KB	~1–2 KB

Post-quantum	Vulnerable (Shor's)	Quantum-resistant (hash-based)	Vulnerable (Shor's)
Used in	Zcash, ZK-Rollups, Dark Forest, Mina	StarkNet, StarkEx	Monero, Grin, Beam

- **ZK-SNARKs:** KZG commitments compress arithmetic circuits into single elliptic-curve elements; SRS from MPC ceremonies (e.g., Powers of Tau) — if the "toxic waste" leaks, false proofs become forgeable. Recursive SNARKs enable Incrementally Verifiable Computation (IVC) / Nova-style accumulation.
- **ZK-STARKs:** FRI tests whether a committed evaluation vector matches a low-degree polynomial via split-and-fold: $f(x)=g(x^2)+xh(x^2)$, folded via verifier challenge α, halving degree each step. No trusted setup, post-quantum resistant, larger proofs.
- **Bulletproofs:** Inner product arguments prove committed values lie in a numeric range without revealing them; no trusted setup, small proofs, but linear verification — suited to range checks (e.g., confidential transactions) rather than large general circuits.

8.6 Circuit Development Stacks

Circom (compiles to R1CS, WASM provers, smart-contract verifiers), Halo2 (PlonKish arithmetization, custom gates/lookup tables, recursive composition without per-circuit trusted setup), Noir and ZoKrates (higher-level DSLs for verifiable privacy-preserving programs).

8.7 Arithmetization Techniques

- **R1CS:** $(L\cdot z) \circ (R\cdot z) = (O\cdot z)$, where $z=(1,x,w)$ is the witness vector and $\circ$ is the Hadamard product.
- **AIR** / **PlonKish:** converts execution traces into polynomial constraints with custom gates and permutation arguments.
- **Polynomial Commitment Schemes:** KZG (pairing $e:G_1\times G_2\to G_T$, SRS $([1]_1,[\tau]_1,\ldots,[\tau^d]_1)$; evaluate via quotient $q(x)=(f(x)-v)/(x-a)$, proof $\pi=[q(\tau)]_1$, single pairing check); IPA/

Bulletproofs (pairing-free, recursive halo-style folding, O(log n) proofs, no trusted setup).

8.8 Advanced Interactive Proofs

- **Multivariate Sum-Check Protocol:** verifies H = Σ...Σ P(x_1,...,x_v) for boolean inputs; Prover sends univariate polynomials $g_i(X_i)$ over v rounds; reduces O(2^v) evaluations to one.
- **GKR Protocol:** interactive verification of layered arithmetic circuits, sublinear in circuit size (O(d·v·log S), d=depth, v=input vars, S=circuit width).

---

9. Post-Quantum Cryptography

9.1 The Quantum Threat

- **Shor's Algorithm:** uses Quantum Fourier Transform to find hidden subgroup periods in O((log N)³) time — breaks factorization and discrete-log systems (RSA, ECC, DH, DSA).
- **Grover's Algorithm:** quadratic speedup on unstructured search, roughly halving symmetric security levels (AES-256 → 128-bit post-quantum security).
- **Harvest Now, Decrypt Later:** adversaries collect ciphertext today to decrypt once quantum computers arrive.
- **Timeline:** Global Risk Institute's 2026 estimate: a cryptographically relevant quantum computer is "quite possible" within 10 years, "likely" within 15.

9.2 NIST Post-Quantum Standards

Finalized (2024–2025): FIPS 203 ML-KEM/CRYSTALS-Kyber (lattice KEM, 3 parameter sets), FIPS 204 ML-DSA/CRYSTALS-Dilithium (lattice signatures), FIPS 205 SLH-DSA (stateless hash-based

signatures), HQC (March 2025, code-based KEM).

Third-round signature candidates (2026): retained multivariate candidates UOV, MAYO, QR-UOV, SNOVA; CROSS and LESS eliminated on security concerns; updated submissions due Aug 14, 2026; standardization conference planned 2027.

9.3 Lattice-Based Cryptography

Hard problems: SVP (shortest nonzero vector), SIVP (n shortest linearly independent vectors), CVP/BDD (closest lattice vector to a target).

Learning With Errors (LWE): given $A \in Z_q^{(m \times n)}$, $b=As+e \bmod q$ (secret s, discrete-Gaussian error e), distinguish b from uniform random. Worst-case to average-case reduction: solving average-case LWE is as hard as worst-case GapSVP on arbitrary lattices.

- **Ring-LWE:** structured variant over $R_q=Z_q[X]/(X^n+1)$ for efficiency.
- **Module-LWE/Module-SIS:** rank-d module formulation balancing security and key size — algebraic basis of ML-KEM and ML-DSA.
- **LLL Algorithm:** polynomial-time basis reduction producing short vectors bounded by $2^{((n-1)/2)} \cdot \lambda_1(\Lambda)$.
- **Lattice Trapdoors / Gaussian Sampling:** constructing A with trapdoor T to efficiently find short x with $Ax=u \bmod q$ — blueprint for IBE and post-quantum signatures.

9.4 Code-Based Cryptography

McEliece Cryptosystem: hardness of decoding random linear codes; uses Goppa codes with hidden structure; very large public keys, fast operations. **HQC** is NIST's selected code-based KEM.

9.5 Hash-Based Signatures

- **Lamport-Diffie one-time signatures:** each signature reveals half

the private key; secure if the hash is collision-resistant.
- **Merkle Signature Scheme:** combines many one-time signatures in a tree. **XMSS** and **LMS** are stateful; **SLH-DSA (SPHINCS+)** is stateless and NIST-standardized.

9.6 Multivariate Cryptography

Based on hardness of solving multivariate quadratic equation systems over finite fields. Candidates: UOV, MAYO, QR-UOV, SNOVA. Advantages: small signatures, small public keys; but recent cryptanalysis has affected several parameter sets.

9.7 Isogeny-Based Cryptography

SIDH (supersingular isogeny DH): very small keys, but **broken in 2022** by the Castryck-Decru attack (dimension-2 isogenies). CSIDH and other isogeny schemes remain under study.

9.8 Quantum Key Distribution (QKD)

- **BB84 (Bennett & Brassard, 1984):** encodes bits in photon polarization across two bases; sifting discards mismatched-basis bits; eavesdropping detected via error-rate estimation; final key via error correction + privacy amplification. Security rests on the Heisenberg uncertainty principle and no-cloning theorem.
- **E91:** uses quantum entanglement (EPR pairs); security via Bell inequality violation.
- **Decoy-state QKD:** counters photon-number-splitting attacks using signal + decoy intensity states.
- **Integration with PQC:** QKD gives information-theoretic key distribution; PQC handles authentication/signatures.
- **Market/deployment:** QKD market projected at USD 13.66B by 2033 (23.2% CAGR); Toshiba achieved 254 km QKD over commercial fiber (April 2025); ID Quantique leads European deployment.

10. Fully Homomorphic Encryption (FHE)

10.1 Concept

Enables computation directly on encrypted data — f applied to ciphertexts yields an encryption of f(plaintext).

10.2 Scheme Taxonomy

- **BGV** / **BFV:** exact modular integer arithmetic over polynomial rings Z_p; modulus switching and relinearization manage key-dimension growth; BFV is a simpler BGV variant.
- **CKKS:** approximate fixed-point complex arithmetic; homomorphic rescaling manages precision; enables privacy-preserving ML inference.
- **TFHE:** operates over the torus T=R/Z (or discrete T$\mathbb{N}$); bit-level Boolean ops (AND/XOR/MUX) as linear ciphertext combinations; fast programmable bootstrapping for arbitrary gates.

10.3 Bootstrapping

Homomorphically evaluates the decryption circuit using a public bootstrapping key to refresh a noisy ciphertext into a fresh one at baseline noise — enabling unlimited-depth computation.

10.4 Noise Growth

Addition adds small noise; multiplication adds significant (sometimes quadratic) noise. Managed via modulus switching and bootstrapping.

10.5 Applications (2026)

Healthcare (cross-institution drug discovery on encrypted data), financial services (credit scoring/fraud detection without sharing raw data), cloud

computing (FHE-as-a-service across jurisdictions), privacy-preserving AI (FHE + MPC + ZKPs combined).

10.6 Libraries/Standardization

Microsoft SEAL (BFV, CKKS, BGV), HElib, PALISADE, Lattigo; NIST and the Homomorphic Encryption Standardization Consortium developing common standards.

10.7 Truth-Table Neural Networks (TT-TFHE)

Converts non-linear activations (ReLU, Sigmoid) into multi-input truth tables evaluated via TFHE lookup tables (e.g., Concrete-Numpy), enabling encrypted inference on datasets like MNIST/CIFAR-10.

---

11. Secure Multi-Party Computation (MPC)

11.1 Definition

Multiple parties jointly compute a function of their private inputs without revealing those inputs to one another; each party may receive a private output.

11.2 Security Models

- **Semi-honest (passive):** adversaries follow the protocol but try to learn extra information.
- **Malicious (active):** adversaries may deviate arbitrarily.

11.3 Fundamental Protocols

- **Yao's Garbled Circuits:** one party garbles a Boolean circuit; the other evaluates it using Oblivious Transfer for their inputs; secure against semi-honest adversaries.
- **Oblivious Transfer (OT):** 1-out-of-2 OT lets a Receiver learn exactly one of the Sender's two messages without revealing which, and without the Sender learning the choice. Foundational to many MPC protocols.
- **GMW Protocol:** secret-shared computation over XOR/AND gates; more communication than garbled circuits, better for arithmetic circuits.
- **BGW** / **BMR:** information-theoretic MPC for an honest majority.

11.4 Secret Sharing

- **Shamir's Secret Sharing:** splits secret s into n shares via polynomial interpolation; any t shares reconstruct, t−1 reveal nothing (Lagrange interpolation over finite fields).
- **Additive Secret Sharing:** $s = x_1+x_2+...+x_n \bmod q$; all shares required to reconstruct.

11.5 Threshold Cryptography

- **NIST Threshold Call (NISTIR 8214C):** solicits threshold scheme proposals across Class N (standardized primitives) and Class S (special primitives) — Signatures (N1/S1), PKE (N2/S2), Symmetric (N3/S3), KeyGen (N4/S4), FHE (S5), ZKPoK (S6), Gadgets (S7). MPTS 2026 hosted 25 preview talks.
- **Threshold signatures:** multiple parties jointly sign; no single party holds the full private key. Used in cryptocurrency custody and distributed key management.

11.6 Recent Research (2026)

Delayed Consistency Check Vulnerability: TPMPC 2026 research found that MPC protocols (Trident, Fantastic Four, SWIFT, Quad) which delay consistency checks allow malicious parties to extract information about intermediate wire values. Fix: replace delayed individual checks

with a single joint consistency check.

---

12. Advanced Cryptographic Primitives

- **Identity-Based Encryption (IBE):** Shamir (1984) proposed it; Boneh-Franklin (2001) gave the first practical pairing-based construction. Public key can be any string (e.g., an email address); requires a trusted Private Key Generator.
- **Attribute-Based Encryption (ABE):** Ciphertext-Policy ABE (policy on ciphertext, attributes on keys) vs. Key-Policy ABE (policy on keys, attributes on ciphertext) — fine-grained access control.
- **Searchable Encryption:** Symmetric Searchable Encryption (SSE) trades security/leakage for search efficiency; Deterministic Encryption (equality checks, leaks equality patterns); Order-Preserving Encryption (OPE, enables range queries, leaks approximate values).
- **Format-Preserving Encryption (FPE):** ciphertext keeps the plaintext's format (e.g., a credit-card-shaped output); NIST standards FF1/FF3-1 (SP 800-38G), based on Feistel networks.
- **Ring Signatures:** signer signs anonymously on behalf of a "ring" of possible signers, unlinkable; used in Monero.
- **Group Signatures:** a group member signs anonymously on the group's behalf; a group manager can revoke anonymity.
- **Blind Signatures:** David Chaum (1982); signer signs a message without seeing its content — digital cash, anonymous credentials, e-voting.
- **Anonymous Credentials:** prove attribute possession without revealing identity; Direct Anonymous Attestation (DAA) used in TPMs.
- **Predicate** / **Functional Encryption:** decryption succeeds only if a hidden predicate holds (predicate encryption); a key sk_f computes $f(m)$ from $Enc(m)$ without revealing m (functional encryption, generalizes IBE/ABE); Inner Product Encryption is a special case computing inner products.
- **Broadcast Encryption & Traitor Tracing:** encrypt for a user

subset with efficient revocation; traitor tracing identifies users who leaked keys.

- **Deniable Encryption:** sender can plausibly deny a message's content; alternate keys yield alternate plausible plaintexts — protects against coercion.
- **Verifiable Delay Functions (VDFs):** require a fixed number of sequential steps to compute but are efficiently verifiable; used in randomness beacons, blockchain consensus, timestamping. Leading constructions: Wesolowski, Pietrzak.
- **Randomness Beacons:** *drand* — distributed, publicly verifiable, unbiased randomness via threshold BLS signatures; used in blockchain, lotteries, protocols.
- **Password Hashing:** must be slow, one-way, GPU/ASIC-resistant, and memory-hard.
    - **Argon2** (2015 Password Hashing Competition winner): Argon2d (GPU-resistant, side-channel-weak), Argon2i (side-channel-safe, weaker vs. GPUs), **Argon2id** (hybrid, recommended).
    - bcrypt (Blowfish-based, adaptive cost, GPU-vulnerable), scrypt (memory-hard, used in Litecoin), PBKDF2 (NIST SP 800-132, iterative but not memory-hard, GPU/ASIC-vulnerable).
    - **Recommendation:** use Argon2id for new systems.
- **Lightweight Cryptography:** for constrained devices (IoT/RFID/sensors). NIST's 2023 competition selected **ASCON** (authenticated encryption + hashing). Other candidates: Grain, Trivium, PRESENT, SIMON, SPECK.
- **White-Box Cryptography:** protects keys in software even under full adversary control of the execution environment (DRM, mobile payments); no provably secure construction exists — all practical schemes have been broken.
- **Steganography & Covert Channels:** hiding messages in innocent-looking media (LSB embedding is simple but detectable; modern approaches use deep learning); covert channels repurpose non-data channels like timing or storage for communication.

---

13. Implementation Security and Side-Channel Resistance

13.1 Side-Channel Attacks

Exploit leakage from physical implementation, not mathematical weakness.

- **Timing attacks:** exploit secret-dependent execution time; mitigated by constant-time code.
- **Cache-based attacks:** Flush+Reload, Prime+Probe (cross-VM/process); **Spectre**/**Meltdown (2018)** exploit speculative/out-of-order execution; **Foreshadow (L1TF)** targets Intel SGX; **ZombieLoad** is another speculative-execution leak.
- **Power analysis:** Simple Power Analysis (direct trace observation), Differential Power Analysis (statistical trace analysis); mitigated via randomization/masking/power-balancing.
- **Electromagnetic analysis:** non-invasive emission measurement, effective against smart cards.
- **Acoustic cryptanalysis:** recovering keys from sounds emitted during decryption (demonstrated against RSA).
- **Fault injection:** laser fault injection, voltage/clock glitching, **Rowhammer** (memory bit-flips from repeated access), **cold boot attacks** (cooling RAM to retain contents after power loss).

13.2 Constant-Time Programming

No branches or memory lookups conditioned on secret data; no variable-time instructions (e.g., division) on secrets; use constant-time conditional move/select operations.

13.3 Memory Safety

Buffer overflows can leak keys or enable code execution; use-after-free can corrupt cryptographic state. Rust's memory safety (without garbage

collection) makes it attractive for cryptographic code.

13.4 Formal Verification

Goal: prove implementations match mathematical specifications. Tools: Coq, Isabelle/HOL, F*, Cryptol. Examples: HACL* (verified library in F*), Fiat-Crypto (verified ECC assembly), partially verified TLS 1.3 implementations.

13.5 Common Implementation Vulnerabilities

- **Nonce reuse (AES-GCM, ChaCha20-Poly1305):** reveals the authentication key, enables forgery; XChaCha20-Poly1305's 192-bit nonce makes random generation statistically safe.
- **Invalid curve attacks:** failing to validate input points lie on the curve reduces operations to small-order subgroups, leaking key material.
- **Low-entropy key generation:** e.g., the 2008 Debian OpenSSL bug, ROCA; always use CSPRNGs.
- **API misuse:** ECB mode, hardcoded keys, improper IVs; misuse-resistant designs (AES-GCM-SIV, NaCl/libsodium) help prevent these.

13.6 CSPRNGs

Required properties: unpredictability, forward secrecy, backtracking resistance. Sources: hardware RNGs (Intel RDRAND/RDSEED, TPM RNGs, quantum RNGs), OS CSPRNGs (/dev/urandom, getrandom(), CryptGenRandom/BCryptGenRandom), and research into chaotic maps (e.g., robust chaotic tent maps tested against NIST 800-22/TestU01).

Historical failures: Dual_EC_DRBG (suspected NSA-backdoored NIST RNG via suspicious parameters), **Debian OpenSSL 2008** (entropy limited to process ID → only 32,768 possible keys).

---

14. Cryptanalysis and Attack Methodologies

14.1 Classical Cryptanalysis

- **Frequency analysis:** exploits non-uniform letter distributions; effective against simple substitution/transposition ciphers (Kerckhoffs' original technique — see §1.4 for his detailed column-based application against polyalphabetic ciphers).
- **Index of Coincidence (IC):** statistical measure of letter-frequency distribution used to analyze polyalphabetic ciphers.
- **Known-plaintext attack:** attacker has matching plaintext/ciphertext pairs.
- **Chosen-plaintext attack (CPA):** attacker chooses plaintexts and obtains their ciphertexts — formalized as IND-CPA (§2.5).
- **Chosen-ciphertext attack (CCA):** attacker chooses ciphertexts and obtains decryptions; CCA2 (adaptive) is the strongest practical model (§2.5).

14.2 Modern Cryptanalytic Techniques

- **Linear cryptanalysis:** linear approximations of non-linear cipher components; needs large known-plaintext samples.
- **Differential cryptanalysis:** studies how input differences propagate; highly effective against DES-era ciphers.
- **Algebraic attacks:** represent the cipher as polynomial equations, solved via Gröbner bases, SAT solvers, or the XL algorithm.
- **Meet-in-the-middle attacks:** build intermediate-value tables to attack double encryption — reduces 2DES's effective security from 2^112 to 2^57 (motivating 3DES).
- **Birthday attacks:** exploit the birthday paradox to find collisions in O(2^(n/2)) instead of O(2^n) — sets the minimum secure hash output size.
- **Length-extension attacks:** affect Merkle-Damgård hashes (MD5, SHA-1, SHA-2) — given H(m) and len(m), one can compute H(m ‖ padding ‖ m') without knowing m. Mitigation: HMAC, or a sponge construction (SHA-3).

14.3 Protocol-Level Attacks

- **Man-in-the-middle:** intercept/modify communications; mitigated by certificate authentication, pinning, PSKs.
- **Replay attacks:** retransmit valid captured messages; mitigated by nonces, timestamps, sequence numbers.
- **Padding oracle attacks:** exploit invalid-padding error signals to decrypt ciphertext (affected SSL/TLS — POODLE, Lucky13); mitigated with AEAD and constant-time padding checks.
- **Downgrade attacks:** force weaker protocol/cipher negotiation; TLS 1.3 removed version negotiation and added explicit downgrade protection.

14.4 Notable Historical Attacks

Attack	Target	Year	Mechanism
Dual_EC_DRBG	NIST RNG	2013	Suspected backdoor via parameter manipulation
Heartbleed	OpenSSL	2014	Buffer over-read leaking memory
POODLE	SSL 3.0	2014	Padding oracle in CBC mode
FREAK	TLS RSA_EXPORT	2015	Downgrade to 512-bit RSA
Logjam	TLS DH_EXPORT	2015	Downgrade to 512-bit DH
DROWN	SSLv2	2016	Cross-protocol attack via SSLv2
SWEET32	3DES/Blowfish	2016	Birthday attack on 64-bit blocks
ROCA	Infineon RSA	2017	Weak prime generation in TPMs

Efail	PGP/S-MIME	2018	Exfiltration via malleability gadgets
Spectre/ Meltdown	CPUs	2018	Speculative-execution side-channels
Raccoon	TLS 1.2 DH	2020	Timing attack on DH key exchange

15. Blockchain and Distributed Systems Cryptography

15.1 Consensus Mechanisms

- **Proof of Work (PoW):** miners solve hash puzzles (Bitcoin: SHA-256, Litecoin: Scrypt); high attack cost but energy-intensive; vulnerable to 51% attacks.
- **Proof of Stake (PoS):** validators lock collateral (Ethereum moved to PoS in 2022, ~99% less energy); fast/efficient but risks stake centralization.
- **Delegated PoS (DPoS):** token holders elect validating delegates (Cosmos, Tron, EOS); scalable but semi-centralized.
- **Practical Byzantine Fault Tolerance (pBFT):** consensus once 2/3 of honest nodes agree (Hyperledger Fabric); efficient with immediate finality but not scalable beyond ~20 nodes; Sybil-vulnerable.
- **Proof of Authority (PoA):** validators stake real-world identity/ reputation (private/consortium chains); fast and accountable but not decentralized.
- **Others:** Proof of Weight (Algorand), Proof of Importance (NEM), Proof of Capacity/Space (Chia, Filecoin).

15.2 Cryptographic Data Structures

- **Merkle Trees:** hash-of-children tree enabling O(log n) inclusion proofs (Bitcoin, certificate transparency, Git).
- **Patricia (Radix) Tries:** efficient key-value store used for Ethereum state.
- **Verkle Trees:** vector-commitment alternative to Merkle-Patricia tries with smaller proofs; proposed for Ethereum scaling.

15.3 Cryptocurrency-Specific Primitives

- **Ring signatures:** obscure the true signer among a ring (Monero).
- **Confidential transactions:** hide amounts using Pedersen commitments + range proofs/Bulletproofs (Monero, Grin, Beam).
- **Multi-signature schemes:** m-of-n signing; **Schnorr MuSig** aggregates into one signature; threshold signatures use distributed key generation.
- **MimbleWimble:** combines confidential transactions with "cut-through" (no addresses, interactive transaction construction) — used by Grin and Beam.

15.4 Smart Contract Security

Reentrancy (recursive external calls draining funds — The DAO, 2016), integer overflow/underflow (largely mitigated in Solidity 0.8+), front-running (mempool transaction-ordering attacks), oracle manipulation (DeFi price-feed attacks).

---

16. Hardware Security and Trusted Computing

- **Hardware Security Modules (HSMs):** dedicated key generation/storage/crypto-operation hardware; FIPS 140-2/140-3 validated; PCIe cards, network-attached, USB tokens; used for CA root keys, payment processing, code signing.
- **Trusted Platform Module (TPM):** standardized secure

cryptoprocessor (ISO/IEC 11889, TCG); secure key storage, attestation, measured boot; TPM 2.0 is current; ROCA (2017) affected Infineon TPMs.
- **Trusted Execution Environments (TEEs):**
  - Intel SGX: encrypted memory enclaves isolated from OS/hypervisor; attacked by Foreshadow (L1TF), Plundervolt, SGAxe, Crosstalk; Intel is deprecating SGX for client CPUs.
  - ARM TrustZone: Secure World / Normal World split; widely used in mobile/IoT/payments.
  - AMD SEV: encrypts VM memory from the hypervisor; attacked by SEVered, CacheWarp.
  - Apple Secure Enclave: dedicated isolated processor handling biometrics, keys, secure boot.
- **Smart Cards:** tamper-resistant embedded microprocessors (SIMs, EMV payment cards, ID cards); standards ISO/IEC 7816, EMVCo; vulnerable to DPA, fault injection, side-channel analysis.
- **Secure Elements:** smaller tamper-resistant chips (hardware wallets like Ledger/Trezor, YubiKeys, IoT); typically Common Criteria EAL5+ certified for financial use.

---

17. Cryptographic Standards and Compliance

17.1 NIST Standards

- **FIPS 140-3** (replaced FIPS 140-2 in 2019): Security Levels 1–4, from software-only to tamper-resistant hardware; tested via CMVP.
- FIPS 197 (AES), FIPS 180-4 (SHA-1/SHA-2), FIPS 202 (SHA-3), FIPS 198-1 (HMAC), FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA).
- SP 800-38 series (block cipher modes), SP 800-56A/B (key establishment), SP 800-57 (key management), SP 800-90A/B/C (random number generation), SP 800-131A (algorithm transitions), SP 800-185 (SHA-3 derived functions: KMAC, TupleHash,

ParallelHash).

17.2 Common Criteria (ISO/IEC 15408)

International security-certification standard; Evaluation Assurance Levels EAL1 (functionally tested) through EAL7 (formally verified, military/nuclear-grade); EAL4 = typical commercial software, EAL5 = smart cards/HSMs, EAL6 = high-security systems.

17.3 Other Standards

ISO/IEC 19790 (aligned with FIPS 140-3), PCI DSS (cardholder data encryption), HIPAA (protected health information encryption), GDPR (recommends encryption/pseudonymization).

17.4 PKI and Certificate Infrastructure

- **X.509 certificates** (RFC 5280): subject, issuer, public key, validity, serial number, extensions; chains from Root CA → Intermediate CA → end entity.
- **Certificate Transparency (CT):** Google-initiated public append-only certificate logs to catch misissuance.
- **OCSP:** real-time revocation checking; OCSP Stapling improves privacy/performance.
- **DNSSEC:** cryptographically signs DNS records (DNSKEY, RRSIG, DS); NSEC/NSEC3 prevent zone enumeration.
- **DANE:** binds TLS certificates to DNS via TLSA records.

17.5 Email Security

DKIM (cryptographic signatures on outgoing mail), DMARC (policy framework built on DKIM+SPF), SPF (authorized-sender DNS records), S/MIME and PGP (end-to-end email encryption — Efail (2018) exploited malleability gadgets in mail clients).

---

18. Socio-Political Dimensions and Cryptographic Ethics

18.1 The Political Nature of Cryptography

In "The Moral Character of Cryptographic Work," Phillip Rogaway argues cryptography is inherently political, not value-neutral: because cryptographic primitives determine who can access, verify, or conceal information, cryptosystem design directly reconfigures power relations among citizens, corporations, and states. This echoes, in modern form, Kerckhoffs' 1883 argument that a cipher's design must not be a point of institutional vulnerability (see §1).

18.2 Surveillance and Privacy

Mass surveillance (automated data collection, traffic analysis, metadata mining) creates power asymmetries that chill speech and weaken democratic institutions. The cypherpunk movement responded with the principle that privacy should be enforced by code, not policy — exemplified by David Chaum's blind signatures for untraceable digital cash and Phil Zimmermann's PGP. Strong end-to-end encryption shifts enforcement power to end users.

18.3 Lawful Access Debates

Law enforcement often argues for mandatory key escrow, exceptional access, or client-side scanning. Security research shows that engineering any backdoor into an encrypted system fundamentally weakens it — the access mechanism itself becomes an exploitable attack surface.

18.4 Technical Alternatives to Backdoors

- **Physical-seizure-constrained decryption:** hardware rate-limiting

requiring extended physical device possession plus a court-authorized secret — supports targeted forensic access while blocking automated mass remote surveillance.
- **Publicly traceable conditional decryption (witness encryption):** every activation of a law-enforcement access capability is permanently, publicly logged, deterring abuse through transparency.

18.5 Export Controls and Human Rights

- **Wassenaar Arrangement:** multilateral export-control regime historically restricting strong-encryption export.
- **Key disclosure laws:** some jurisdictions compel decryption/password disclosure; "rubber-hose cryptanalysis" refers to coercion-based key extraction (not a technical attack) — deniable encryption is a technical countermeasure.
- **Human rights:** UN Special Rapporteurs have affirmed encryption and anonymity as essential privacy protections underpinning other human rights.

18.6 Responsible Disclosure

Cryptographic vulnerabilities can have global impact; coordinated disclosure balances public safety against the need to patch (e.g., Heartbleed, ROCA, Dual_EC_DRBG were disclosed through coordinated — if imperfect — processes).

---

19. Authoritative Research Platforms and Learning Resources

19.1 Research Blogs

Platform / Author	Focus	Key Contributions
A Few Thoughts on Cryptographic Engineering (Matthew Green)	Protocol flaws, ZKPs, hardware backdoors, provable security	ROM breakdowns, ZKP primers, Dual_EC_DRBG post-mortems
ImperialViolet (Adam Langley)	TLS/SSL internals, ECC, memory safety	X.509 parsing vulnerabilities, constant-time routines, TLS 1.3 optimization
Schneier on Security (Bruce Schneier)	Foundational theory, security architecture, policy	System threat models, Kerckhoffs-compliant advocacy
Trail of Bits Research	Code audits, ZKP circuits, PQC, side-channels	Static analysis tools, ZK constraint audits, compiler-level flaw detection
Cloudflare Research Blog	Applied PQC, OHTTP, TLS, ECC, distributed randomness	ECDSA vs EdDSA guides, PQ KEM deployment, drand beacons
David Wong's Cryptography Blog	ECC arithmetic, MPC, Noise, ZK-SNARKs	Mathematical breakdowns of applied proof systems
NCC Group Research Blog	Cryptanalysis, protocol bugs, hardware attacks	Side-channel leakage identification, non-standard curve cryptanalysis

19.2 Video / Course Resources

- **Dan Boneh** — Stanford CS255 / Online Cryptography: perfect secrecy proofs, provable security reductions, number theory.
- **Christof Paar** — Introduction to Cryptography: LFSRs, Trivium, ARX ciphers, Galois field arithmetic, DPA.
- **Silvio Micali** — MIT 6.875 Foundations of Cryptography: OWFs,

Goldreich-Levin, Blum-Micali PRGs.
- **Yael Kalai** — MIT 6.5630 Advanced Topics: interactive proofs, sum-check protocol, GKR protocol.
- **Brynmor Chapman** — MIT 6.1200J Lecture 10: Euclidean/ Extended Euclidean algorithms, modular exponentiation.
- **UC Berkeley RDI ZK MOOC** (Boneh, Thaler, Zhang, Goldwasser, Song): R1CS/AIR/PlonKish, KZG/IPA, recursive SNARKs.
- **ZK Whiteboard Sessions** (ZK Hack/Polygon): Plonky2, FRI, prover acceleration, lattice-based PQ SNARKs.
- **Alfred Menezes** — Mathematics of Lattice-Based Cryptography: lattice geometry, LLL, SVP/SIVP/CVP, M-LWE/M-SIS.
- **Chris Peikert & Oded Regev** — Advanced lattice seminars: LWE formulation, lattice trapdoors, Gaussian sampling.
- **Chalk Talk PQ Series:** Shor's algorithm/QFT mechanics, LWE noise-flooding visualization.
- **FHE.org** / **Zama seminars** (Paillier, Gentry, Benamira): TFHE, bootstrapping, BGV/BFV/CKKS, TT-TFHE.
- **Computerphile** / **Andrea Corbellini:** elliptic curve visualizations, ECDSA mechanics.
- **IACR Archive (@TheIACR):** 4,100+ recorded talks from Eurocrypt, Crypto, Asiacrypt, CHES, Real World Crypto.

---

20. Strategic Outlook and Future Directions

- **Post-quantum migration:** Canada's roadmap targets initial migration plans by April 2026, priority transitions by 2031, full migration by 2035; CISA released an initial PQC-supporting hardware/software category list (January 2026). Organizations should start with cryptographic inventory and risk assessment, then migrate the most vulnerable systems first.
- **Privacy-preserving convergence:** combining FHE, MPC, and ZKPs enables computation verifiably correct without revealing inputs or intermediate results — a defense-in-depth approach for sensitive AI applications.
- **Formal verification and correctness:** ongoing focus on formally verified crypto code and misuse-resistant primitive designs to

close the gap between mathematical proofs and real-world implementation bugs.
- **Cryptographic agility:** the ability to swap algorithms quickly matters for responding to cryptanalytic breaks, migrating to PQC, and meeting evolving standards — via regular key rotation and protocols that support secure algorithm negotiation with downgrade protection.
- **The persistent operational gap:** mathematical security proofs (§2.5) routinely fail to account for implementation-level side channels, low-entropy key derivation, and API misuse — driving continued investment in formal verification, misuse-resistant design, trust-minimizing ZK paradigms, constant-time implementations, and security auditing.
- **Final synthesis:** cryptography sits at the intersection of mathematics, engineering, and social power. Its trajectory depends on mathematical rigor, engineering excellence, standardization, ethical commitment to privacy, quantum readiness, privacy-preserving computation, and decentralization of trust (MPC, threshold cryptography, blockchain) — the same tension between provable strength and practical, accountable deployment that Kerckhoffs first articulated in 1883 (§1).

---

21. Glossary of Key Terms

Term	Meaning
AEAD	Authenticated Encryption with Associated Data
CCA / CPA	Chosen-Ciphertext / Chosen-Plaintext Attack
CSPRNG	Cryptographically Secure Pseudo-Random Number Generator
DH / DLP	Diffie-Hellman / Discrete Logarithm Problem
EU-CMA	Existential Unforgeability under Chosen-Message Attack (signature/MAC security)

ECC / ECDH / ECDSA	Elliptic Curve Cryptography / Diffie-Hellman / Digital Signature Algorithm
FHE	Fully Homomorphic Encryption
HMAC	Hash-based Message Authentication Code
IBE	Identity-Based Encryption
IND-CPA / IND-CCA	Indistinguishability under Chosen-Plaintext / Chosen-Ciphertext Attack
IV	Initialization Vector
KDF / KEM	Key Derivation Function / Key Encapsulation Mechanism
LWE	Learning With Errors
MAC / MPC	Message Authentication Code / Multi-Party Computation
NIST	National Institute of Standards and Technology
Nonce	Number used once
OWF	One-Way Function
PPT	Probabilistic Polynomial-Time (the standard model of an "efficient" adversary)
PQC	Post-Quantum Cryptography
PRF / PRG / PRP	Pseudo-Random Function / Generator / Permutation
QKD	Quantum Key Distribution
ROM	Random Oracle Model
RSA	Rivest-Shamir-Adleman
SIS	Short Integer Solution
SNARK / STARK	Succinct / Scalable (Transparent) Argument of Knowledge
TEE	Trusted Execution Environment
TLS	Transport Layer Security

ZKP	Zero-Knowledge Proof

22. Recommended Reading and References

Foundational texts: Katz & Lindell, *Introduction to Modern Cryptography* (2nd ed.) — the primary source for the formal-definitions/provable-security framework in §2.5; its four-part structure (classical ciphers → private-key crypto → number theory → public-key crypto, plus appendices on mathematical background and algorithmic number theory) is a natural next step for readers who want the full proofs behind the summaries in §2–§6 above. Also: Goldreich, *Foundations of Cryptography* (Vols. 1–2); Boneh & Shoup, *A Graduate Course in Applied Cryptography*; Smart, *Cryptography Made Simple*; Paar & Pelzl, *Understanding Cryptography*.

Standards: NIST FIPS 140-3, FIPS 197, FIPS 203–205; RFC 8446 (TLS 1.3); RFC 8439 (ChaCha20-Poly1305).

Key papers: Goldwasser, Micali & Rackoff, "The Knowledge Complexity of Interactive Proof Systems" (1985); Bellare & Rogaway, "Random Oracles are Practical" (1993); Canetti, Goldreich & Halevi, "The Random Oracle Methodology, Revisited" (1998); Gentry, "Fully Homomorphic Encryption Using Ideal Lattices" (2009); Ben-Sasson et al., "Scalable, transparent, and post-quantum secure computational integrity" (2018).

Historical primary source: Kerckhoffs, A. "La Cryptographie Militaire," *Journal des Sciences Militaires*, Vol. IX (Jan–Feb 1883) — origin of Kerckhoffs' Principle (§1 above). Original scanned text: http://petitcolas.net/kerckhoffs/crypto_militaire_1.pdf

Online resources: Cryptography Engineering Blog (Matthew Green), ImperialViolet (Adam Langley), Schneier on Security (Bruce Schneier), Trail of Bits Research Blog, IACR ePrint Archive, NIST Computer Security Resource Center.